PROG2002 - Web Development II

Assignment 3: A Complete Dynamic Website

Student ID: 24832054 Last Name: Gao First Name: Chengyu

Team: Jie Bao (24831941) and Chengyu Gao (24832054) - 2 members

Title of the project: Giving Coast - Charity Events on the Gold Coast

Live client site: https://24832054.it.scu.edu.au/

Live admin site: https://24832054.it.scu.edu.au/admin/

GitHub repository: https://github.com/BaoJie001/PROG2002-A3 (private, marker invited)

1. Introduction / Motivation

Assessment 2 produced a working dynamic website. Three pages - Home, Search and Event details - read charity events out of a MySQL database through a small Express API and rendered them in the browser. It was a real client/server application, but it could only read. A member of the public who saw an event they liked had no way to book a place, and a staff member who needed to correct a venue or cancel a washed-out fun run had to open phpMyAdmin and edit a row by hand.

Assessment 3 closes both of those gaps. The public site gains a registration page and an event page that shows who has already booked. The staff site is new: it lists every event regardless of status and supports the full create, read, update and delete cycle. The database gains a registrations table, the API moves from read-only to full CRUD, and both websites plus the API are deployed to SCU cPanel so they can be reached from the internet rather than only from localhost.

We also went past the minimum in three places, because the marking rubric rewards them and because they made the project more interesting to build. Both websites are Angular 22 applications written in TypeScript rather than plain DOM code; the event page shows a live weather forecast for the venue, fetched from the Open-Meteo API;

and there is a Postman collection with assertions so that every endpoint can be re-checked in one click. Section 8 records that extra work separately so it can be marked as such.

2. Problem Statement

The case study is unchanged from Assessment 2: four small Gold Coast charities - a running club, a community kitchen, a music school and a wildlife shelter - run a steady stream of fundraisers and need one place to publish them. What changes is that the site now has to accept data as well as display it, and that it has to survive being deployed on shared hosting. That raises three problems that are not obvious from the requirement list.

Modelling a booking when there is no user account. The brief says there is no USER table, and it also says that one person may register for many events but only once per event. Both have to be true at the same time. If a person's only identity is their email address, then the uniqueness rule is really a rule about the pair (event, email). It has to be enforced by the database rather than by a check in JavaScript, because two requests that arrive within the same millisecond can both pass a JavaScript check and both insert a row.

Deleting an event without breaking the register. The rule 'an event can only be deleted if it has received no registrations' is a statement about data integrity, not about the interface. It has to hold even when somebody calls the API directly with curl, and it has to hold when a booking arrives at the same moment as a delete. Hiding the Delete button on the admin screen is a courtesy to the user, not a guarantee to the database.

One build, three addresses. On cPanel the public site is served from the root of the domain, the admin site from /admin/, and the API from a Node application mounted at /api. A single-page application normally assumes it owns the root of the URL space: it loads its scripts from /main.js and it takes over every path so that its own router can decide what to show. Both assumptions are false here, and the deployment has to correct them.

3. Solution

The site keeps the three-layer shape from Assessment 2 and adds a second client to it. Nothing in the browser ever talks to MySQL; everything goes through the API.

Table 1 - The four parts of the system.

Layer	Technology	Responsibility
Database	MySQL 8.0	charityevents_db: organisations, categories, events, registrations. Owns the integrity rules (foreign keys, unique key, CHECK constraints).
Server	Node.js 20+ / Express 4.21 / mysql2 3.11	The only process with database credentials. Validates input, runs parameterised SQL, maps results to JSON, decides status codes, and proxies the weather API.
Public client	Angular 22.2 / TypeScript 6	Home, Search, Event details and Register pages. Calls the API with HttpClient and renders the result with signals.
Staff client	Angular 22.2 / TypeScript 6	Event list, new event, edit event. Same API, different endpoints (it may also write).

A page request always follows the same path. The Angular router matches a URL and creates a component; the component asks a service for data; the service calls the API with HttpClient and returns a Promise; Express receives the request, validates the parameters and runs one parameterised query through the mysql2 connection pool; MySQL returns rows; Express serialises them to JSON; the service resolves the Promise; the component writes the result into a signal; and the template, which is already bound to that signal, re-renders. No part of that chain knows about the layer after it, which is why the same API can serve two different front ends and why we could replace the A2 front end without touching a single SQL statement.

Concretely, a booking works like this. On /event/5 the visitor presses Register. The button links to /register?eventId=5 - a query string, not a JavaScript variable, so the page can be refreshed, bookmarked or shared and still knows which event it is for. The register page reads eventId from the URL, fetches the event so it can show the name, date, venue and price, and fills in the ticket total live as the visitor changes the quantity. On submit it POSTs to /api/registrations. The API validates the body, checks that the event exists and is still accepting bookings, inserts the row, and answers 201 with the new record. The page shows a confirmation and a link back to the event, and the event page now lists the new booking first because it sorts by registration date descending.

4. Data Schema

Two changes were needed: a new table to hold bookings, and two columns on events so that a weather forecast can be requested for the right place. Everything else from Assessment 2 was left alone, because it already modelled the case study correctly.

Table 2 - The registrations table.

Column	Type	Why it exists
registration_id	INT AUTO_INCREMENT PK	Surrogate key. The natural key would be (event_id, email), which is two wide columns; a single integer is cheaper to index and easier to reference.
event_id	INT NOT NULL, FK -> events	Which event was booked. ON UPDATE CASCADE, ON DELETE RESTRICT.
first_name / last_name	VARCHAR(80) NOT NULL	Stored separately from email so the staff screen can sort and greet by name, and so a shared family address still reads correctly.
email	VARCHAR(150) NOT NULL	Contact detail, and part of the uniqueness rule.
phone	VARCHAR(30) NOT NULL	The charities ring people when a event is cancelled - an email sent the night before is often missed.
tickets	INT NOT NULL DEFAULT 1	How many places were bought. CHECK (tickets > 0 AND tickets <= 50).
registration_date	DATE NOT NULL	The day the visitor says they booked. This is a business value, not a timestamp, which is why it is a DATE.
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	When the row was actually written. Kept separate from registration_date so the two can never be confused when auditing.
notes	VARCHAR(255) NULL	Dietary requirements, accessibility needs, 'arriving late'.

The constraints are where the design actually lives, so they are worth spelling out:

```
CONSTRAINT fk_registration_event
    FOREIGN KEY (event_id) REFERENCES events (event_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,

CONSTRAINT uq_registration_event_email
    UNIQUE (event_id, email),

CONSTRAINT chk_tickets_positive CHECK (tickets > 0),
CONSTRAINT chk_tickets_sane     CHECK (tickets <= 50),
CONSTRAINT chk_email_shape      CHECK (email LIKE '%_@%_._%')
```

The UNIQUE key is the answer to the first problem in Section 2: 'one person, one booking per event' becomes a property of the table, so it holds no matter how many requests arrive at once, and the API only has to translate the resulting duplicate-key error into a friendly 409. ON DELETE RESTRICT is the answer to the second problem: MySQL itself refuses to delete an event that still has bookings, so the rule

survives a careless DELETE typed into phpMyAdmin. The API checks first and returns a helpful message, but the constraint is the thing that guarantees it.

Two values are deliberately not stored. The total cost of a booking is tickets multiplied by the event's ticket_price, and the number of places still available is the event's capacity minus the sum of tickets sold. Both are derived when they are needed. Storing them would create a second copy of the truth that could drift out of step the first time a price changed.

On the events table, latitude and longitude were added as DECIMAL(9,6), which is accurate to about 11 centimetres - far more than a weather grid cell needs, and enough that the same columns could later be used to draw a map. Nullable, because not every venue has been geocoded yet, and the weather panel has to cope with that.

The sample data was extended to 16 registrations spread across five events, comfortably over the required minimum of 10, so that the 'who is going' list on the event page and the registration list on the admin edit screen both have something realistic to show. The whole schema and dataset still export to a single SQL file, database/charityevents-db.sql, as the brief requires.

5. API Design

The API exposes eleven endpoints plus a health check. They are grouped by resource rather than by page, so the shape of the URL says what is being acted on:

Table 3 - The endpoints. Every one of them is called by at least one page.

Method	Path	Purpose	Success	Errors
GET	/api/events	Upcoming events, optionally filtered by date, location, category and free-text search	200	400
GET	/api/events/past	Past events, for the 'previously' section of the home page	200	-
GET	/api/events/:id	One event plus its complete registration list, newest booking first	200	400, 404
POST	/api/events	Create an event (admin)	201	400
PUT	/api/events/:id	Replace an event's details (admin)	200	400, 404
DELETE	/api/events/:id	Delete an event, blocked when bookings exist	200	400, 404, 409
GET	/api/registrations	All bookings, or those for one event via ?event_id=	200	400
POST	/api/registrations	Create a booking (public site)	201	400, 404, 409

GET	/api/categories	Categories with their icon and banner image	200	-
GET	/api/organisations	The four charities and their contact details	200	-
GET	/api/weather/:eventId	Forecast for the event's venue, proxied from Open-Meteo	200	400, 404, 502
GET	/api/health	Reports the API version and whether the database answers	200	500

Worked example 1 - POST /api/registrations

Purpose. Creates a booking. This is the endpoint behind the Register button, and it is the only write endpoint the public site can reach.

Request body (JSON):

```
{
  "event_id": 5,
  "first_name": "Amelia",
  "last_name": "Ward",
  "email": "amelia.ward@example.com",
  "phone": "0412 345 678",
  "tickets": 2,
  "registration_date": "2026-10-20",
  "notes": "One vegetarian meal please"
}
```

Responses:

- **201 Created.** The row was inserted. The body returns the stored record including its new registration_id and the event's ticket price, and a Location header points at it.
- **400 Bad Request.** A field is missing or malformed - an empty first name, an email with no @, tickets of 0 or 51, or a date that is not a real calendar date. The body names the field and gives the reason, and the form shows that message under the offending input.
- **404 Not Found.** No event has that event_id.
- **409 Conflict.** Three different situations, each with its own message: this email has already booked this event; the event has been suspended; or the event has already taken place. Each one is a genuine conflict with the current state of the resource rather than a malformed request, which is exactly what 409 is for.

Worked example 2 - DELETE /api/events/:id

Purpose. Removes an event, but only when doing so destroys no bookings. The endpoint counts the registrations for the event first; if the count is not zero it returns 409 with the number of bookings in the message, so the admin screen can say 'this event has 4 bookings and cannot be deleted' rather than just failing. If the count is zero it deletes and returns 200. As noted in Section 4, the ON DELETE RESTRICT foreign key means the same rule is enforced even if a future version of this endpoint forgets to check.

Why these methods, and why these status codes

GET is used for every read, POST to create, PUT to replace an entire event, and DELETE to remove one. PUT rather than PATCH because the admin form always submits the whole event - every field is on the screen, pre-filled - so there is no such thing as a partial update in this application, and pretending otherwise would add a second code path for no benefit. The distinction that matters most is 400 versus 409: a 400 means 'your request was wrong, change it and try again', while a 409 means 'your request was fine, but the world is not in a state where it can be carried out'. A duplicate booking is the second kind; sending 400 for it would have told the visitor to fix something that was not broken.

Validation is not duplicated between the two write endpoints. A shared module, api/routes/validate.js, holds the field rules for an event and for a booking, so a rule can only be changed in one place and the public site and the staff site receive identical messages. Where the rule is also a database constraint, that is deliberate duplication of a different kind: the application check produces a good message, the constraint produces a guarantee.

Every query is parameterised. There is no string concatenation of user input anywhere in the codebase, LIMIT and OFFSET are coerced to bounded integers, the JSON body parser is capped at 100 kB, and CORS is an explicit allow-list of the two deployed origins rather than a wildcard. Credentials live in environment variables on the server and never reach the browser; the Angular environment files contain only the base URL of the API.

The weather endpoint is a proxy, on purpose

GET /api/weather/:eventId does not return the Open-Meteo response to the browser. It looks up the event's latitude and longitude, calls Open-Meteo from the server, translates the numeric weather codes into the plain descriptions the brief lists (0 -> clear sky, 61 -> light rain, and so on), and returns a compact object with the next few days. Three reasons: the browser never has to know that a third party exists, so the upstream URL can change without a rebuild; the mapping from code to words is testable JavaScript on the server rather than a lookup table repeated in a template; and when Open-Meteo is unreachable the API can answer 502 and the page says plainly that there is no forecast, instead of showing an empty box.

6. Client-side Website

Four pages, all Angular 22 components with their template, styles and TypeScript in separate files: Home, Search, Event details and Register. Each route carries a title, so the browser tab and the back button both say something meaningful.

Event details. The page calls GET /api/events/:id once and receives the event and its bookings together, so there is no second round trip and no chance of the two disagreeing. The 'Who is going' panel lists each booking with the name, the number of tickets and the date it was bought, in the order the API returned them - newest first, exactly as the brief asks. The Register button is a router link to /register?eventId=, so it works with the keyboard, with the back button and with a right-click 'open in new tab'.

Register. A Reactive Form with seven fields matching the table in Section 4. Validation runs as the visitor types and again on submit; each field shows its own message rather than one generic alert, and the submit button is disabled until the form is valid. The ticket quantity has a live total underneath it, computed from the event price, so the visitor sees what they are committing to before they commit. The booking date defaults to today and cannot be in the future. On success the form is replaced by a confirmation panel with the booking reference and a link back to the event, which is where the visitor naturally wants to go next.

Search. The filters from Assessment 2 (date, location, category, free text) are unchanged in behaviour but now run against the same endpoint the admin list uses, and the page keeps the chosen filters in the URL so a search result can be shared.

Weather panel. On the event page, a small panel shows the forecast for the venue over the next few days. It is a separate request from the event itself, so a slow or failing weather service never delays or breaks the page.

7. Admin-side Website

A separate Angular application, deployed under /admin/, with three screens and a persistent navigation bar. Authentication is out of scope per the brief, so the site opens straight onto the event list.

- **List.** Every event, whatever its status, with the name, date, venue, category, price, tickets sold, amount raised against the goal, and status. Filter chips switch between all, active, suspended and past, and a search box narrows by name. The status filter is a query parameter on GET /api/events rather than a filter applied in the browser, so the list stays correct however large the table grows.
- **New.** The same form component as the edit screen, empty. Required fields are marked and validated; the Save button calls POST /api/events and returns to the list with the new event highlighted.
- **Edit.** The form pre-filled from GET /api/events/:id, with that event's bookings listed underneath - name, email, tickets, date - and a total ticket count at the top, so staff can answer 'how full is this event?' without leaving the screen. Save calls PUT.
- **Delete.** A button on each row in the list. It asks for confirmation, then calls DELETE. If the API answers 409 the screen reports exactly why ('this event has 4 bookings and cannot be deleted') instead of showing a generic failure.

The navigation bar is part of the shell component, not of any page, so it is present on every screen by construction rather than by remembering to paste it into each template - which is also why a 404 route can redirect to the list without losing the menu.

8. Work Beyond the Minimum

The brief makes Angular and the weather API optional but rewards them, so this section records them separately from the required work.

Table 4 - Work that goes beyond the required feature list.

Extra	What it involved	Where to look
Angular 22 for the public site	Standalone components with no NgModules, signals and computed values instead of manual DOM wiring, HttpClient with firstValueFrom so every call returns a Promise, typed models for every API response, lazy route titles.	clientside/src/app/
Angular 22 for the staff site	A second application sharing the same conventions: Reactive Forms with custom validators, a shared form component reused by the new and edit screens, route parameters for :id, an interceptor-free service layer.	adminside/src/app/
Open-Meteo weather API	latitude/longitude added to the schema, a server-side proxy endpoint, weather codes mapped to text, graceful degradation when the upstream call fails.	api/routes/weather.js
Postman collection with assertions	Every endpoint in a runnable collection, including the 409 cases, so the API can be regression-tested in one click.	docs/postman/
One-click local start	start-all.bat runs the API and both Angular dev servers together, so the team and the marker do not have to remember three commands.	start-all.bat
Character-set hardening	Both the connection pool and the exported SQL file now pin utf8mb4, so the emoji icons survive a move to a host whose default is the three-byte character set.	api/event_db.js, database/

9. Interface Storyboard

Before the two websites were built, the seven screens were storyboarded on the template supplied with the unit: one page per screen, with the layout sketched, the interface text listed, and the media noted. Each storyboard page carries four frames - GRAPHICS, TEXT, AUDIO/VIDEO and NOTES - over the five-row table the template provides (Description, Content, Interactions, Media Creation, Navigation).

The seven screens are the four public pages and the three staff pages. Table 5 maps each one to the endpoint it depends on; Figures 1 to 7 show them as they are running on the deployed site, with the storyboard row for each screen reproduced underneath.

Table 5 - The seven screens, and the endpoint each one depends on.

Screen	Page	What it has to do	Calls
S1	Home	Make a visitor want to attend, and reach an event in one click	GET /api/events, GET /api/events/past

S2	Search	Narrow by date, location and category, any combination	GET /api/events?date=&location=&category=, GET /api/categories
S3	Event details	Everything about one event, plus who is already going	GET /api/events/:id, GET /api/weather/:eventId
S4	Register	Collect the booking and confirm it	POST /api/registrations
S5	Staff - event list	Every event whatever its status, with the doors to create, edit and delete	GET /api/events?status=all&period=all
S6	Staff - new event	Create an event with validation	POST /api/events, GET /api/organisations, GET /api/categories
S7	Staff - edit event	Change an event, and see who it affects	GET /api/events/:id, PUT /api/events/:id

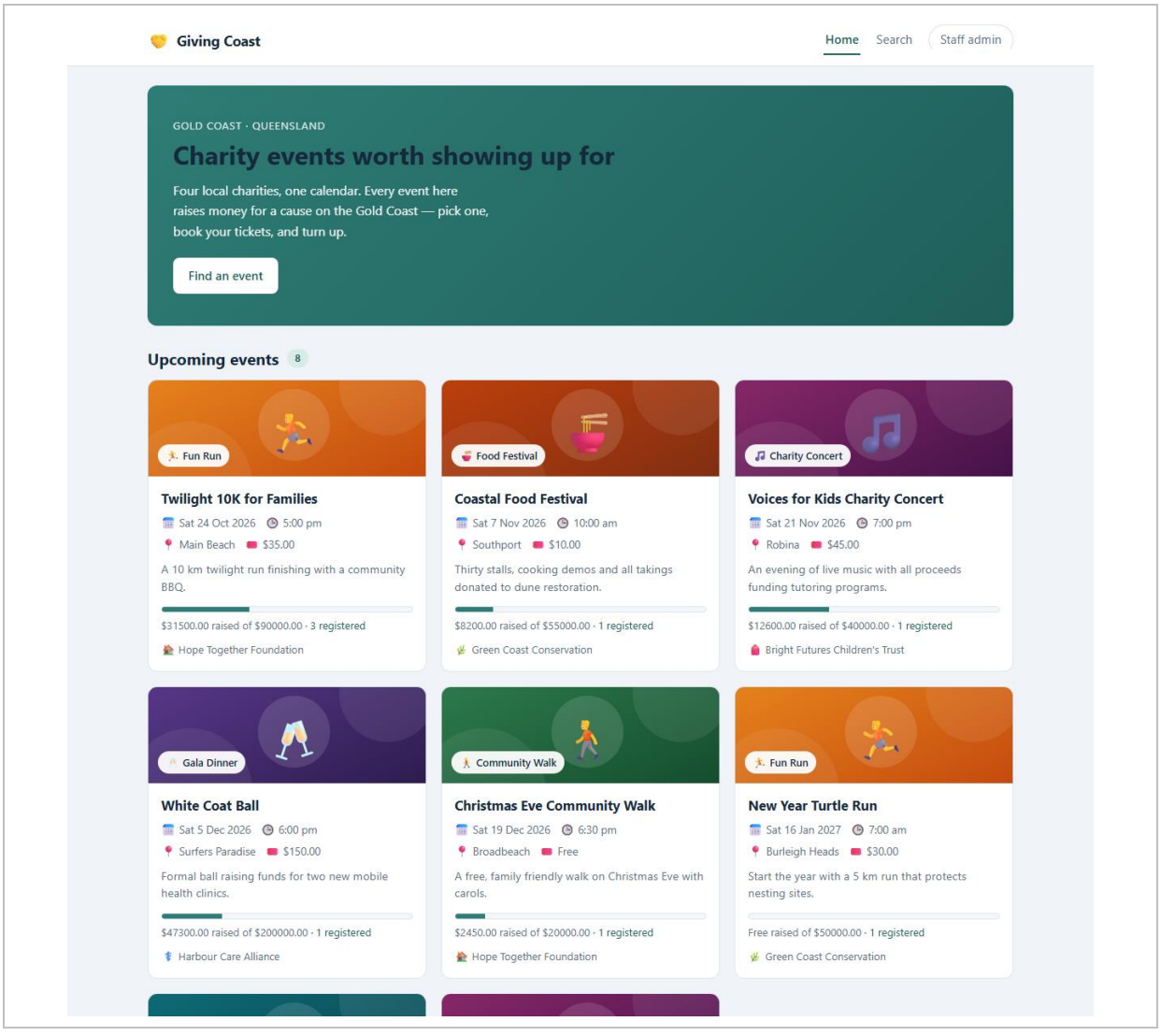


Figure 1 - S1: Home

Graphics: hero panel, a grid of eight event cards, then the recently finished grid. Text: eyebrow 'Gold Coast - Queensland', headline 'Charity events worth showing up for',

the 'Find an event' button, and on every card the name, date, time, venue, price, '$X raised of $Y goal' and 'N registered'. Media: none - the category icons are characters stored in the database, and the colour bands and progress bars are CSS computed from API values. Navigation: the hero button and the menu go to Search, any card opens S3.

Note: A broken back end shows immediately as an empty grid with a retry button rather than a silent blank page.

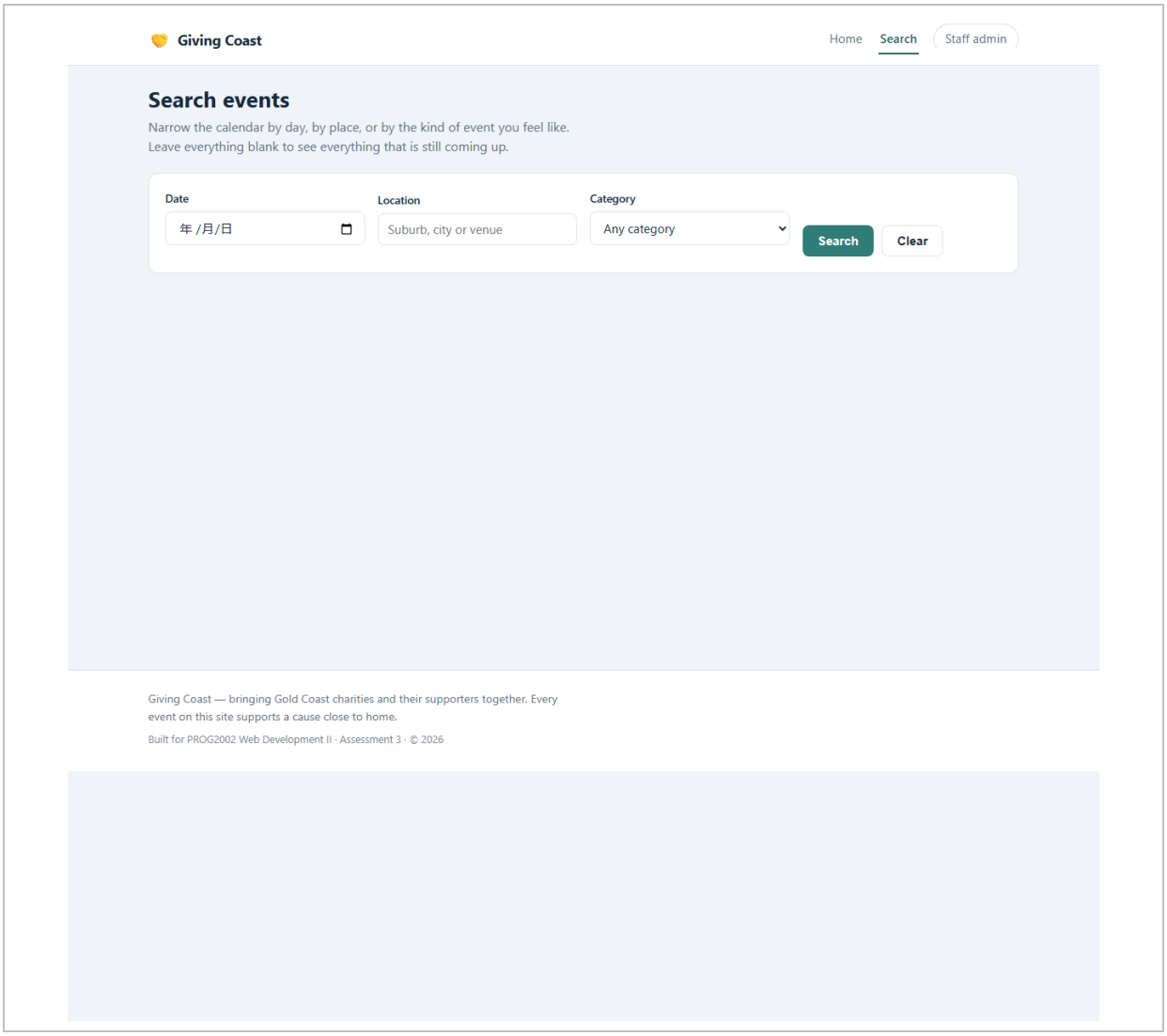


Figure 2 - S2: Search

Graphics: a filter bar over the same card grid the home page uses, so the two pages cannot look or behave differently. Text: 'Leave everything blank to see everything that is still coming up', labels Date / Location / Category, buttons Search and Clear. Media: the category dropdown is filled from GET /api/categories. Navigation: reached from the home hero or the menu; a card opens S3.

Note: Filtering runs on the server, so this page only draws results. 'No search yet' and 'no results' are deliberately two different states.

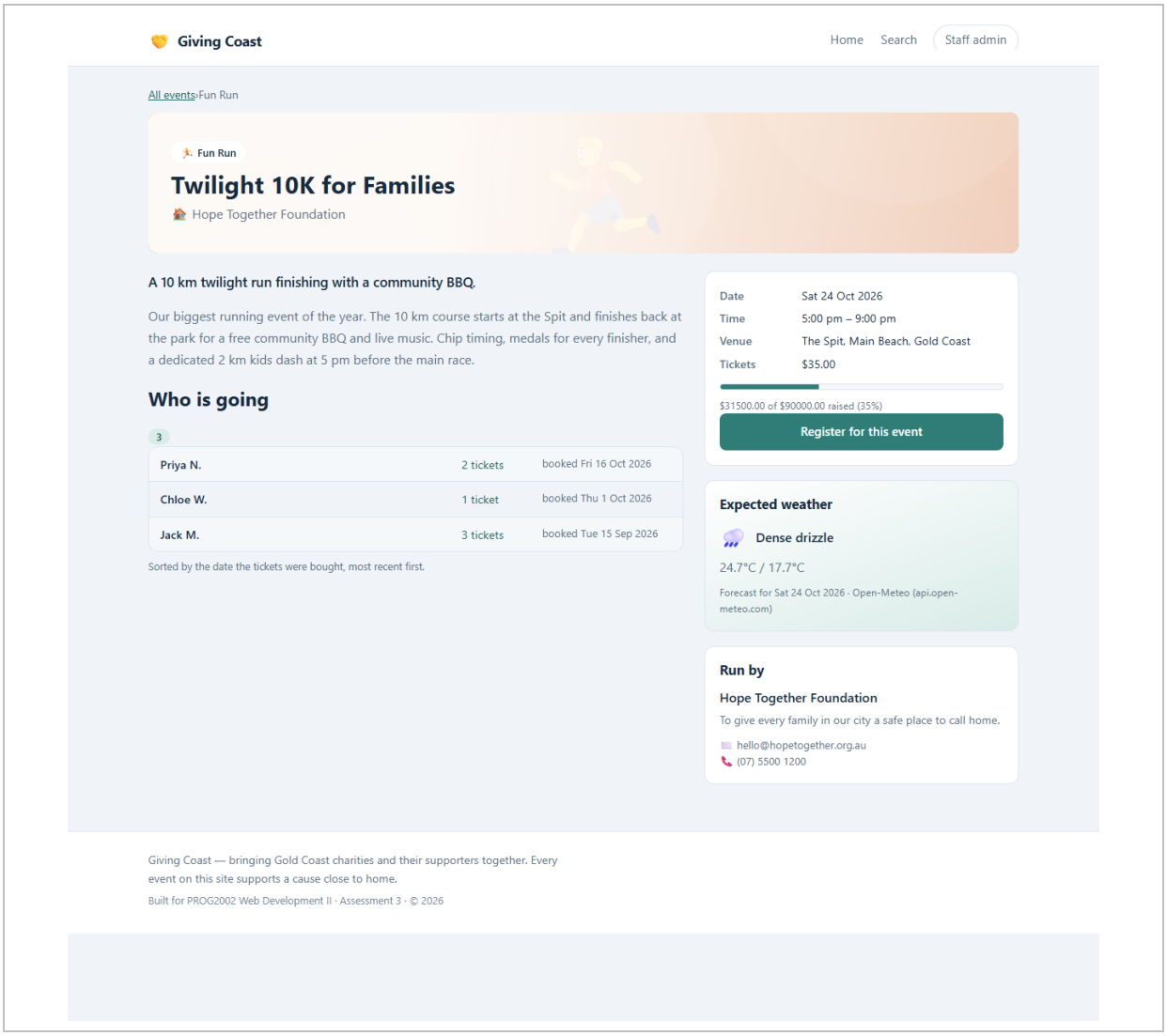


Figure 3 - S3: Event details

Graphics: breadcrumb, the event block, the progress bar, then three panels - who is going, expected weather, and who runs it. Text: 'Who is going' with each name, ticket count and booking date; 'Expected weather' with the icon, description and max/min; the Register button. Media: the weather icon is a character mapped from the Open-Meteo code by our own API, not an image file. Navigation: in from any card; out to S4 through the Register button.

Note: Social proof sits next to the price because it is the strongest reason to book. The weather is a separate request, so a slow third party never delays the page the visitor came for.

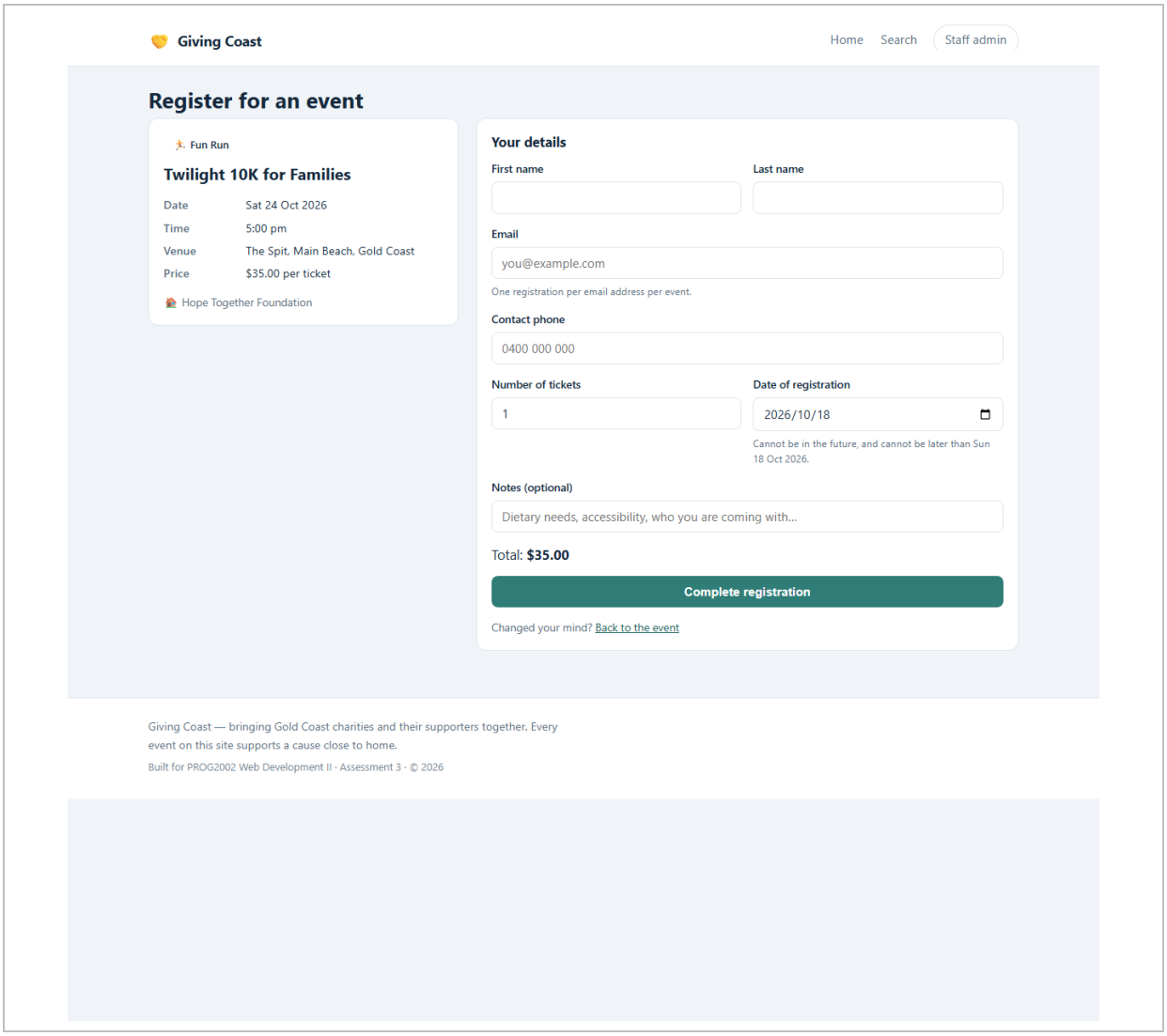


Figure 4 - S4: Register

Graphics: two columns - the event on the left, the form on the right. Text: 'Your details', then first name, last name, email, contact phone, number of tickets, date of registration and optional notes; a live 'Total: $X'; the 'Complete registration' button. On success the form is replaced by 'You are registered'. Media: none - the total is computed from tickets times price. Navigation: in from S3 carrying eventId in the query string; after booking, back to S3.

Note: Drawing this screen changed it: a single column became two, because a visitor needs to see the event while filling in the form, not afterwards.

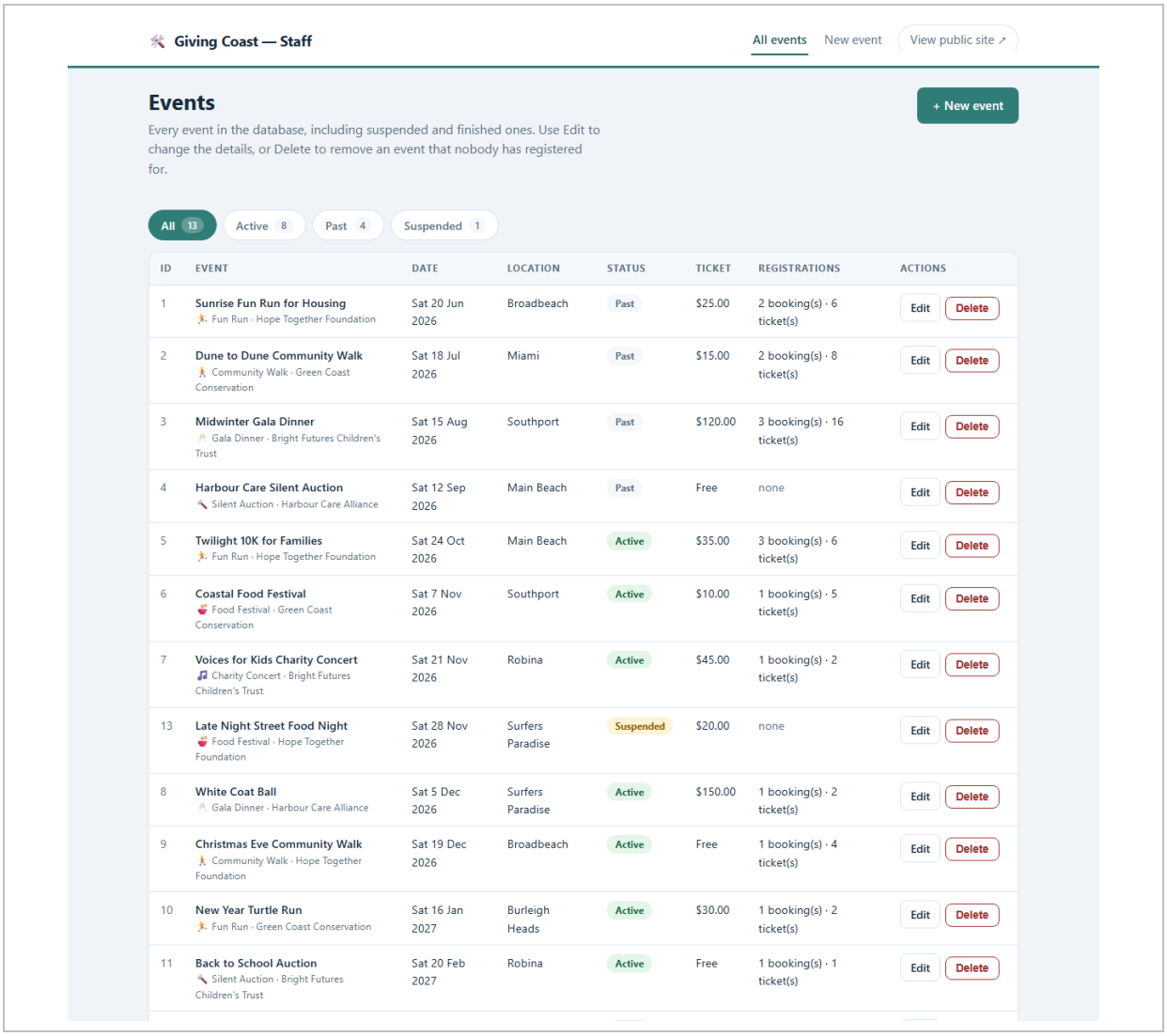


Figure 5 - S5: Staff - event list

Graphics: a status filter strip above a table with an action column. Text: '+ New event', chips reading All 13 / Active 8 / Past 4 / Suspended 1, column headings ID / Event / Date / Location / Status / Ticket / Registrations / Actions, and Edit and Delete on each row. Media: status badges are coloured text. Navigation: in from the public menu; out to S6, S7, or the public site.

Note: The counts live on the chips because 'how many are suspended?' is the first question staff ask, and they are counted from the rows actually on screen.

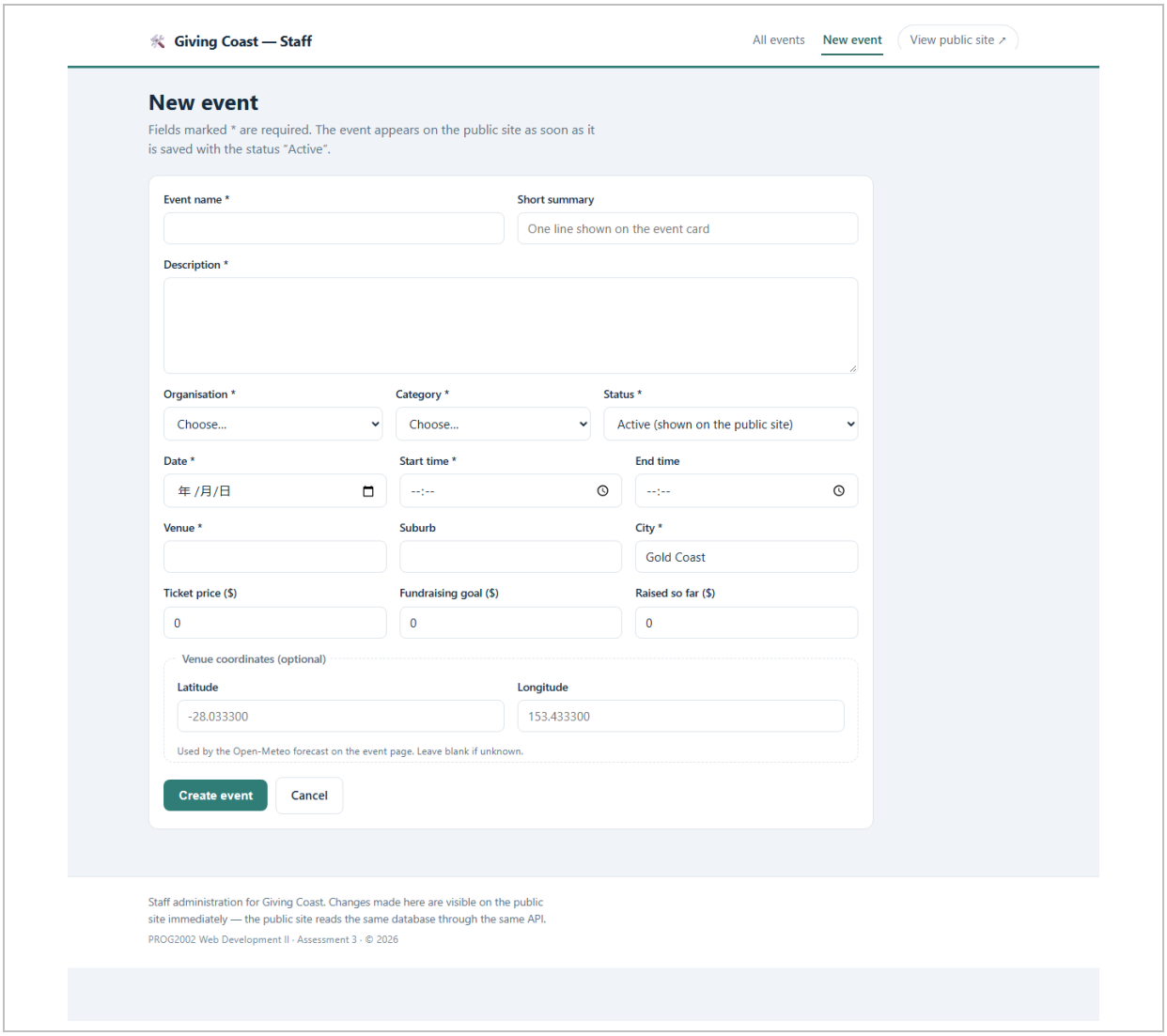


Figure 6 - S6: Staff - new event

Graphics: one long form in three groups, ending with a separate Venue coordinates group. Text: required fields marked with an asterisk, and the note 'Used by the Open-Meteo forecast on the event page. Leave blank if unknown.' Media: the two dropdowns are filled from the same lists the public site shows. Navigation: in from S5; Save or Cancel returns to S5.

Note: Create and edit share one form component, so the two field sets cannot drift apart.

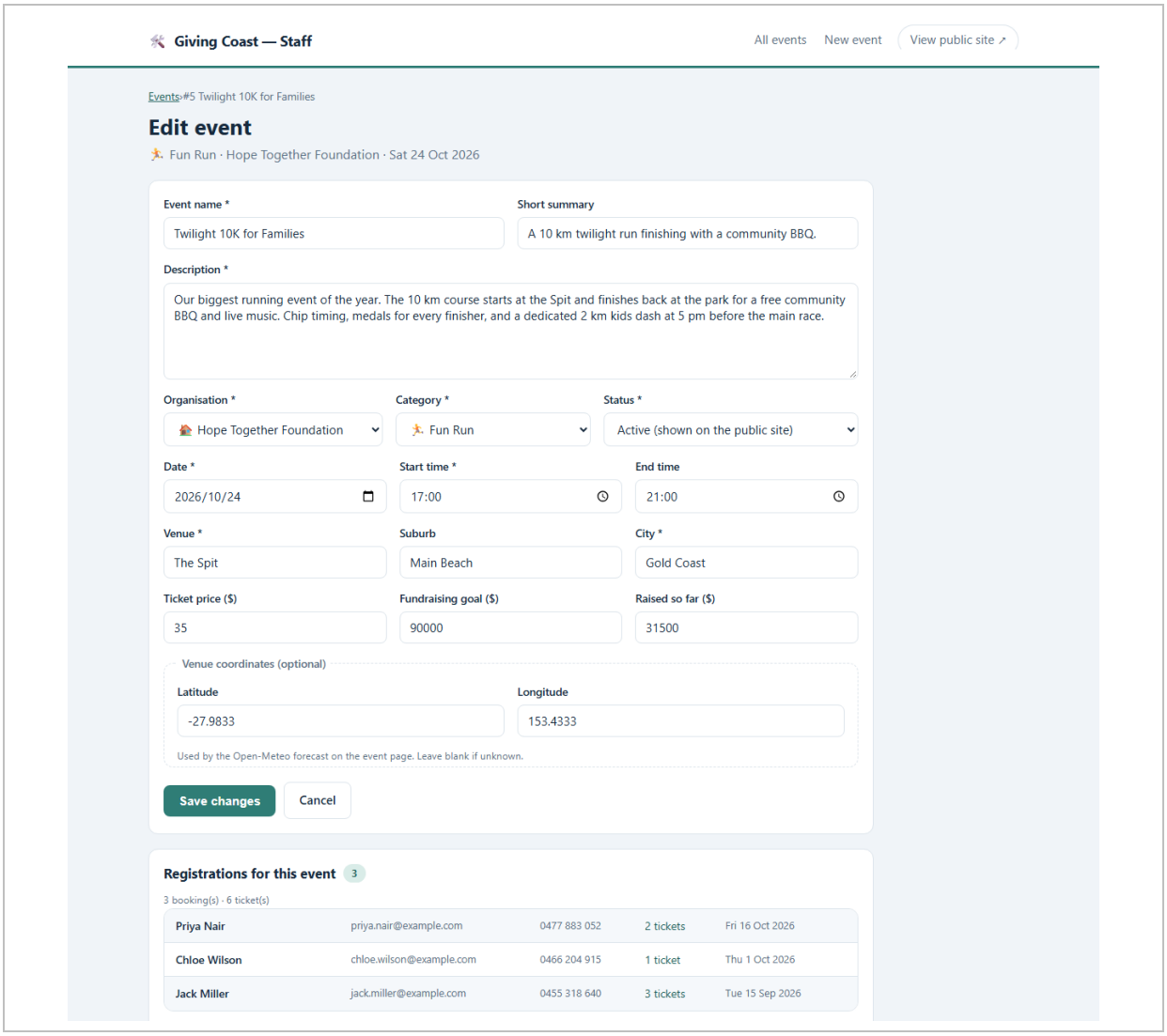


Figure 7 - S7: Staff - edit event

Graphics: the same form, pre-filled, with the registration list underneath. Text: the booking count and total tickets, then each booking with name, email, phone and date, and the sentence 'Because registrations exist, this event cannot be deleted - suspend it instead.' Media: none beyond the data. Navigation: in from S5; Save or Cancel returns to S5.

Note: This is where drawing the screens changed the design again: the delete rule moved out of a help page and onto this screen, where it is read at the moment it matters.

Writing the TEXT frame also caught a wording problem before any code existed: the first draft of the register page said 'Purchase tickets' while several events are free. Drawing the screen made that inconsistency visible in a way that reading the requirements had not.

10. Deployment

Each team member deployed the whole stack to their own cPanel account, as the brief requires. The two deployments are identical in structure and differ only in the database name and the credentials.

Table 6 - The two live deployments.

Item	Jie Bao	Chengyu Gao
Public site	https://24831941.it.scu.edu.au/	https://24832054.it.scu.edu.au/
Admin site	https://24831941.it.scu.edu.au/admin/	https://24832054.it.scu.edu.au/admin/
API health	https://24831941.it.scu.edu.au/api/health	https://24832054.it.scu.edu.au/api/health
Database	jbao14_charityevents	cgao15_charityevents

The steps were: create the database and user in cPanel; import the exported SQL through phpMyAdmin; upload the API folder and register it as a Node.js application with server.js as the startup file, mounted at /api; run npm install; put the database credentials and the allowed origin into the application's environment; build each Angular app with ng build --base-href set for the directory it would live in and upload the output to public_html and public_html/admin; and add a small .htaccess to each directory that rewrites unknown paths back to index.html so the Angular router, rather than the web server, decides what a deep link means.

The full procedure, including the exact commands and the problems below, is written up in docs/deployment.md so it can be repeated from scratch.

11. Problems Encountered and What They Taught Us

Four things went wrong during deployment. They are recorded here because each one turned out to be a lesson about the layer beneath the one we thought we were working on.

Every emoji became a question mark. The category and organisation icons (a runner, a bowl, a music note, a house) appeared correctly on localhost and as ? on the server. Our first guess was the SQL import, so we added SET NAMES utf8mb4; it did not help. Our second guess was the driver, so we pinned charset: 'utf8mb4' in the mysql2 pool; necessary, but still not enough. The real cause was one level lower: the CREATE TABLE statements never declared a character set, so the tables inherited the database default, and the host's default was utf8mb3 - three bytes per

character, which cannot hold a four-byte emoji at all. It had been failing silently since the tables were created, which is why no error was ever raised. The proof was the medical symbol on the wildlife shelter: it sits inside the three-byte range and it was also corrupted, which ruled out 'emoji' as the category of the bug and pointed at the whole character set. The fix was to convert the tables to utf8mb4 and rewrite the affected values; the lasting fix was to pin the character set in the export so the same thing cannot happen again.

The admin site was blank. It worked locally and served a white page on the server, with no 404 and no error message. An Angular build assumes it is mounted at the root unless it is told otherwise, so index.html asked the browser for /main.js while the file had been uploaded to /admin/main.js. The browser received the HTML page again - the .htaccess rewrite - and tried to parse HTML as JavaScript, which fails without a visible message. Rebuilding with --base-href=/admin/ fixed it. The lesson is that a rewrite rule can hide a broken asset path by answering 200 to everything.

Environment variables that were never saved. The API answered 'Access denied for user root@localhost', which is the error you get when no credentials are found and the defaults are used. The variables had been typed into the cPanel form but had not been applied to the application. We switched to a .env file read by dotenv, which is checked into neither the repository nor the zip, and restarted the application: the health endpoint then reported the correct database name. A health endpoint that reports which database it is actually connected to was what made this diagnosable in seconds rather than hours, and we would add one to any API from the start now.

A dependency install that failed once and then worked. Run NPM Install returned a generic error on the first attempt on both accounts and succeeded unchanged on the second. Shared hosts time out under load. We kept a pre-built node_modules archive as a fallback and documented the retry rather than pretending it does not happen.

12. Teamwork

The work was split so that each of us owned a vertical slice of the stack and could explain it end to end in the video, rather than one person writing all the SQL and the other all the HTML.

Table 7 - Division of work.

Part of the brief	Jie Bao (24831941)	Chengyu Gao (24832054)
1. Database	Registrations table, venue coordinates, sample data	Reviewed the schema; booking-date rule
2. REST API	Event CRUD, connection pool, shared validation, delete rule	Registration endpoint, weather proxy, categories and organisations
3. Public site	Angular shell and data layer, Home, Search	Event details, Register page, weather panel
4. Staff site	Event list, delete flow	New and edit screens, shared form component
5. Deployment	Its own cPanel; wrote the deployment guide	Its own cPanel; followed and corrected the guide
Docs and testing	README, deployment notes, video plan, GenAI declaration	Postman collection with assertions, code comments

Git history is the evidence. Each commit message is prefixed with the owner and the part of the brief it belongs to - A1, A2, A3 for Jie's work and B1, B4, B7, B9, B11, B12 for Chengyu's - so the log can be read as a checklist against Table 7. The A3 and B3 commits are deliberately interleaved rather than grouped by author: each person's work was pushed as it was finished, and a commit describes one change rather than one afternoon. Work spans more than one day, and every commit message explains why the change was made, not just what changed.

To keep the repository honest when one of us could not push from their own machine, we agreed that commits are only ever attributed to the person who wrote the code, and we verified that with git log --pretty='%an %ae' before submitting. The teammate's commits carry his own name and address, not a shared account.

13. My Individual Contribution

My half of the project was everything that happens after somebody decides to attend: the registration endpoint, the registration form and the event page that shows who is coming, the weather feature, and both staff screens for creating and editing an event. I also deployed the whole stack to my own cPanel account.

The registration endpoint. I wrote POST /api/registrations, which looks simple and is not. A booking can be refused for three quite different reasons - the event has been suspended, the event has already taken place, or this email address has already booked this event - and all three are answered with 409 Conflict, because none of them is a malformed request; the request is fine, the world is not in a state where it can be carried out. The same method also validates eight fields before it

touches the database, and it still catches MySQL error 1062 at the end: two people submitting the same address in the same instant can both pass my duplicate check, so the unique key gets the last word and the endpoint turns that error into the same friendly 409.

Event page and registration form. On the event details page I built the 'Who is going' panel, which lists every booking newest first, and the Register button, which passes the event through the URL rather than through a JavaScript variable - so the page survives a refresh, a bookmark or a shared link. The registration form is a Reactive Form with a message under each field instead of one generic alert, and a live ticket total so the visitor sees the cost before they commit. Two things I fixed after testing: the booking date can no longer be later than the event itself, and the ceiling is re-checked in the submit handler, because a max attribute on an input is a hint to the browser and not a guarantee to the server.

Weather and the smaller endpoints. I built the Open-Meteo integration as a proxy on our own server: the browser asks our API, our API asks Open-Meteo, maps the numeric weather codes to plain descriptions, and returns only the few values the panel needs. That keeps the third party out of the browser, keeps the code-to-words mapping testable, and means that when Open-Meteo is unreachable the page can say there is no forecast instead of showing an empty box. I also wrote the categories and organisations endpoints.

Staff site. I built the new-event and edit-event screens on one shared form component, so the two cannot drift apart. The edit screen loads the event together with its bookings and shows a total ticket count, so staff can see how full an event is without leaving the page. I also added the status filter to the list, with counts taken from the rows actually on screen, so a filter button can never claim a number the table does not show.

Deployment and testing. I deployed everything to my own cPanel account following the guide, which is where I hit the problem described in Section 11: the API reported 'Access denied for root@localhost' because the environment variables had not been saved, and we moved to a .env file to make the configuration explicit. That error message was what taught me to read a connection failure carefully - it named the user and the host, and the answer was in those two words. I also wrote the Postman

collection with assertions, including the cases that must fail, so the whole API can be re-checked in one click.

The video follows the same division: I demonstrate the files I own and explain the reasoning behind the decisions in them, rather than reading the code aloud.

14. Conclusion

Assessment 3 turned a read-only catalogue into an application that accepts data, protects it, and can be operated by somebody who is not a programmer. The parts we are most satisfied with are the ones where a rule lives in exactly one place: the uniqueness of a booking is a UNIQUE key, the deletion rule is a foreign key, and the validation messages come from a single shared module. The parts that taught us most were the failures, all of which happened one layer below where we were looking - a character set in a CREATE TABLE, a base href in a build, an environment variable that was never saved.

If we were to continue, the next steps would be authentication for the staff site, an email confirmation on booking, and pagination on the event list, which is the first thing that will strain once the table grows past a few hundred rows.

15. GenAI Acknowledgement and Chat Log

I acknowledge that I have used GenAI tools to complete this assessment. I used WorkBuddy (an AI coding assistant) to brainstorm the schema change needed for event registration and the set of REST endpoints that would serve both websites; to check grammar and clarity in code comments, the README and the deployment notes; to format documentation; and to review my own code for bugs, within the parameters outlined in the Assessment Brief and by the Unit Assessor.

Table 8 - How GenAI was used, and what it did not do.

Where GenAI was used	What it contributed	What we did ourselves
Brainstorming	Candidate table shapes for registrations; the list of endpoints to consider	Chose UNIQUE (event_id, email); rejected a separate attendee table; chose 409 over 400
Code review	Pointed out that toISOString() returns a UTC date and shifts the calendar day; that the delete check and the delete itself must not be separated by other work	Reproduced both, wrote the fixes, tested them against the live API
Writing support	Grammar and clarity in comments and documentation; table formatting	All design decisions, all final wording, and the structure of the

		code
Debugging	Suggested character-set causes to rule out during the emoji investigation	Ran the experiments that eliminated the first two explanations and identified the real one

Every design decision recorded in this report is mine and I can explain it, and I am prepared to demonstrate any file I own line by line if asked. The chat log of the sessions is kept with the submission, and a fuller statement is in GENAI-DECLARATION.md in the repository.

Appendix A - Where Each Requirement Is Met

Table 9 - Requirement to evidence.

Requirement	Evidence
New table related to events, with keys	database/charityevents-db.sql - registrations, fk_registration_event, uq_registration_event_email
At least 10 registrations	16 rows across 5 events in the same SQL file
event_db.js updated	api/event_db.js - pool with utf8mb4, transactional helper
Endpoint returning an event with its registrations	GET /api/events/:id - api/routes/events.js
Insert, update and delete endpoints	POST /api/events, PUT /api/events/:id, DELETE /api/events/:id
Delete blocked when bookings exist	409 response plus ON DELETE RESTRICT in the schema
Registration page with validation and confirmation	clientside/src/app/pages/register/
Event page lists bookings, newest first	clientside/src/app/pages/event-detail/, ORDER BY registration_date DESC, registration_id DESC
Register button links to the registration page	Router link to /register?eventId= in event-detail.html
Admin list of all events regardless of status	adminside/src/app/pages/event-list/ with ?status=all
Admin create, update, delete with feedback	event-new, event-edit, event-list
Navigation on every page	adminside/src/app/app.html - shell component
Both websites and the API deployed	Table 6; /api/health returns database: connected
GitHub history with meaningful messages	29 commits, two authors, prefixed with owner and part
Angular used for both websites (extra)	Table 4
Interface storyboard for all seven screens	Section 9 and storyboard-giving-coast.docx
Open-Meteo weather API (extra)	api/routes/weather.js and the panel on the event page

Body text (excluding tables): approximately 4,100 words.